

Final Project:

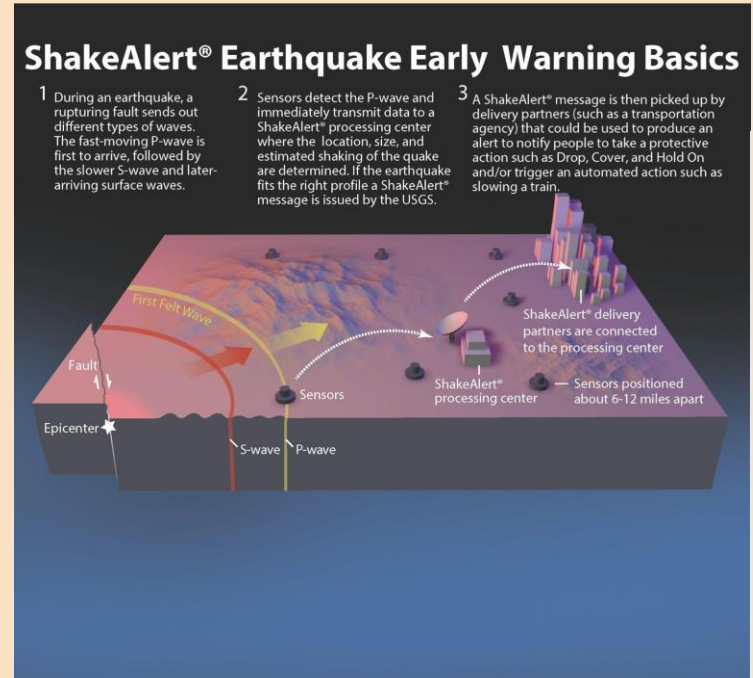
Earthquake Alert Prediction Model

AGENDA	Introduction	1
	Motivation	2
	Dataset Features	3
	Initial Analysis	4
	First Model: Decision Tree	5
	Results I	6
	Improving the Model	7
	Second Model: Random Forest	8
	Results II	9
	In Comparison	10

Earthquake Alert System

- Not about “predicting” earthquakes
- Sensors can detect early signs of earthquakes
- Data collected from those sensors is used to issue an appropriate alert for residents in the area
- **Aim:** Predicting the correct alert label

<https://www.usgs.gov/media/images/earthquake-early-warning-basics-0>



MOTIVATION

“Whether or not Earthquake Early Warning systems can provide useful (i.e., timely and correct) alerts before earthquake shaking begins at a site depends on numerous factors. The hardest hit sites tend to be the ones that are the most difficult to alert because they are typically located close to the epicenter, where the seismic waves arrive first. Sites that are farther away from the epicenter can potentially receive longer warning times, but in these places the alerts may be less important because the shaking is weaker.

In this study we run three prominent Earthquake Early Warning algorithms retrospectively on 219 of the largest earthquakes in Japan. We analyze how often and under what circumstances the algorithms can provide useful warnings. We put special emphasis on the challenging sites that receive high shaking intensities. **We find that all algorithms can alert a significant fraction even of these difficult cases.**”

Meier, M., Kodera, Y., Böse, M., Chung, A., Hoshiba, M., Cochran, E., Minson, S., Hauksson, E., & Heaton, T. (2020). How Often Can Earthquake Early Warning Systems Alert Sites With High-Intensity Ground Motion? *Journal of Geophysical Research: Solid Earth*, 125(2). <https://doi.org/10.1029/2019jb017718>

DATASET



AHMED MOHAMED ZAKI · UPDATED 2 MONTHS AGO



50



Code



Download



Earthquake Alert Prediction Dataset

Balanced Seismic Data for Machine Learning in Earthquake Alert Prediction



Data Card

Code (23)

Discussion (0)

Suggestions (0)

About Dataset

Earthquake Alert Prediction Dataset

A comprehensive preprocessed earthquake dataset containing **1,300 samples** specifically engineered for machine learning applications in seismic hazard assessment and earthquake alert prediction systems. This dataset has been enhanced through advanced oversampling techniques using SMOTE to achieve optimal class balance, ensuring superior model training performance for earthquake alert classification tasks.

Usability ⓘ

10.00

License

Attribution 4.0 International (CC ...)

Expected update frequency

Annually

<https://www.kaggle.com/datasets/ahmeduzaki/earthquake-alert-prediction-dataset/data>

DATASET FEATURES

1

Magnitude

The Magnitude of the earthquake (Richter Scale)

6

Alert

Categorical earthquake alert level classification (Target Variable)



2

Depth

The depth where the earthquake begins to rupture (km)

3

CDI

The maximum reported intensity for the event range

4

MMI

The maximum estimated instrumental intensity for the event

5

Sig

A comprehensive significance measure determined by multiple factors including the other features

Higher values indicate more significant seismic events

Can be negative

INITIAL ANALYSIS

```
df = pd.read_csv("earthquake_alert_balanced_dataset.csv")  
df.head()
```

Python

	# magnitude	# depth	# cdi	# mmi	# sig	🚩 alert
0		7.0	14.0	8.0	7.0	0.0 green
1		6.9	25.0	4.0	4.0	-33.0 green
2		7.0	579.0	3.0	3.0	-13.0 green
3		7.3	37.0	5.0	5.0	65.0 green
4		6.6	624.0	0.0	2.0	-98.0 green

5 rows x 6 cols 10 ▾ per page

⏪ < Page 1 of 1 > ⏩



```
df.isnull().sum()  
# df.isna().sum()
```

#	0
magnitude	0
depth	0
cdi	0
mmi	0
sig	0
alert	0

6 rows x 1 cols 10 ▾ per page

df.dtypes

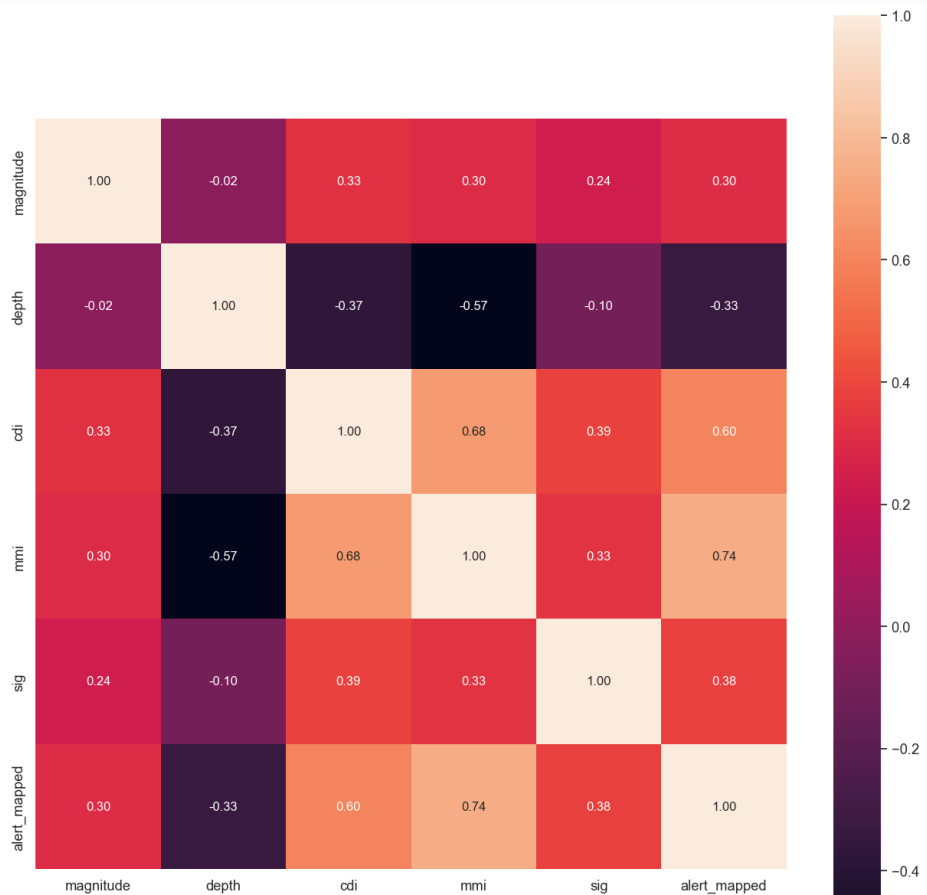
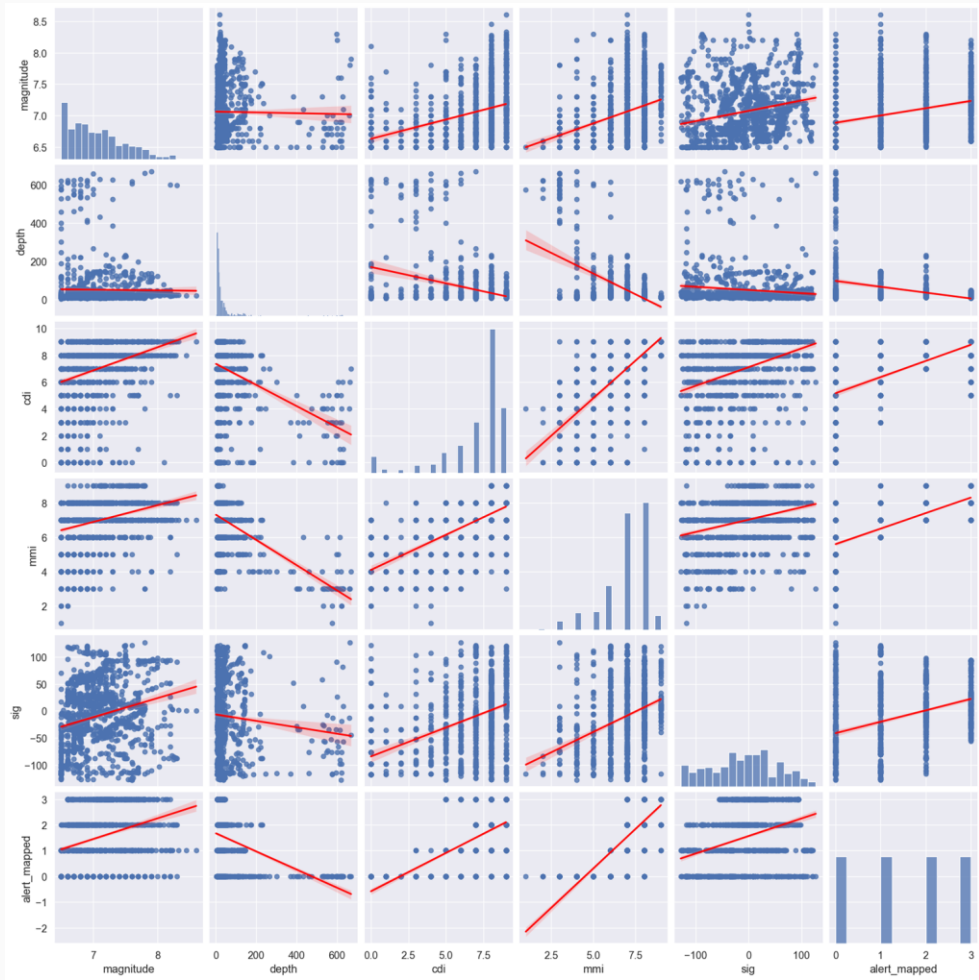
	0
magnitude	float64
depth	float64
cdi	float64
mmi	float64
sig	float64
alert	object

6 rows x 1 cols 10 ▾ per page

df.shape

(1300, 6)

INITIAL ANALYSIS



First Model: Decision Tree

```
class Node():
    def __init__(self, feature=None, threshold=None,
                  left=None, right=None, gain=None, value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.gain = gain
        self.value = value
```

✓ 0.0s

Python

```
class DecisionTree():
    def __init__(self, min_samples=2, max_depth=4):
        self.min_samples = min_samples
        self.max_depth = max_depth

    def split_data(self, dataset, feature, threshold):
        left_dataset = []
        right_dataset = []
        for row in dataset:
            if row[feature] <= threshold:
                left_dataset.append(row)
            else:
                right_dataset.append(row)
        return np.array(left_dataset), np.array(
            right_dataset)

    def entropy(self, y):
        entropy = 0
        labels = np.unique(y)
        for label in labels:
            label_e = y[y == label]
            p = len(label_e)/len(y)
            entropy += -p * np.log2(p)
        return entropy
```

```
def information_gain(self, parent, left, right):
    information_gain = 0
    parent_entropy = self.entropy(parent)
    weight_left = len(left) / len(parent)
    weight_right = len(right) / len(parent)
    entropy_left, entropy_right = self.entropy(left),
        self.entropy(right)
    weighted_entropy = weight_left * entropy_left +
        weight_right * entropy_right
    information_gain = parent_entropy - weighted_entropy
    return information_gain
```

```
def best_split(self, dataset, num_samples,
               num_features):
    best_split = {'gain': -1, 'feature': None,
                  'threshold': None}
    for feature_index in range(num_features):
        feature_values = dataset[:, feature_index]
        thresholds = np.unique(feature_values)
        for threshold in thresholds:
            left_dataset, right_dataset = self.split_data(
                dataset, feature_index, threshold)
            if len(left_dataset) and len(right_dataset):
                y, left_y, right_y = dataset[:, -1],
                    left_dataset[:, -1], right_dataset[:, -1]
                information_gain = self.information_gain(y,
                    left_y, right_y)
                if information_gain > best_split["gain"]:
                    best_split["feature"] = feature_index
                    best_split["threshold"] = threshold
                    best_split["left_dataset"] = left_dataset
                    best_split["right_dataset"] = right_dataset
                    best_split["gain"] = information_gain
    return best_split
```

```
def calculate_leaf_value(self, y):
    y = list(y)
    most_occurring_value = max(y, key=y.count)
    return most_occurring_value

def build_tree(self, dataset, current_depth=0):
    X, y = dataset[:, :-1], dataset[:, -1]
    n_samples, n_features = X.shape
    if n_samples <= self.min_samples and current_depth <= self.max_depth:
        best_split = self.best_split(dataset, n_samples,
            n_features)
        if best_split["gain"] and len(best_split) == 5:
            left_node = self.build_tree(best_split
                ["left_dataset"], current_depth+1)
            right_node = self.build_tree(best_split
                ["right_dataset"], current_depth+1)
            return Node(best_split["feature"], best_split
                ["threshold"], left_node, right_node, best_split
                ["gain"])
        leaf_value = self.calculate_leaf_value(y)
        return Node(value=leaf_value)

def fit(self, X, y):
    dataset = np.concatenate((X, y), axis=1)
    self.root = self.build_tree(dataset)

def predict(self, X):
    preds = []
    for x in X:
        pred = self.make_prediction(x, self.root)
        preds.append(pred)
    return np.array(preds)

def make_prediction(self, x, node):
    if node.value != None:
        return node.value
    else:
        feature = x[node.feature]
        if feature <= node.threshold:
            return self.make_prediction(x, node.left)
        else:
            return self.make_prediction(x, node.right)
```

291

✓ 0.0s

Python

Decision Tree: Results + Improvement

Base model parameters: min_samples=2, max_depth=4

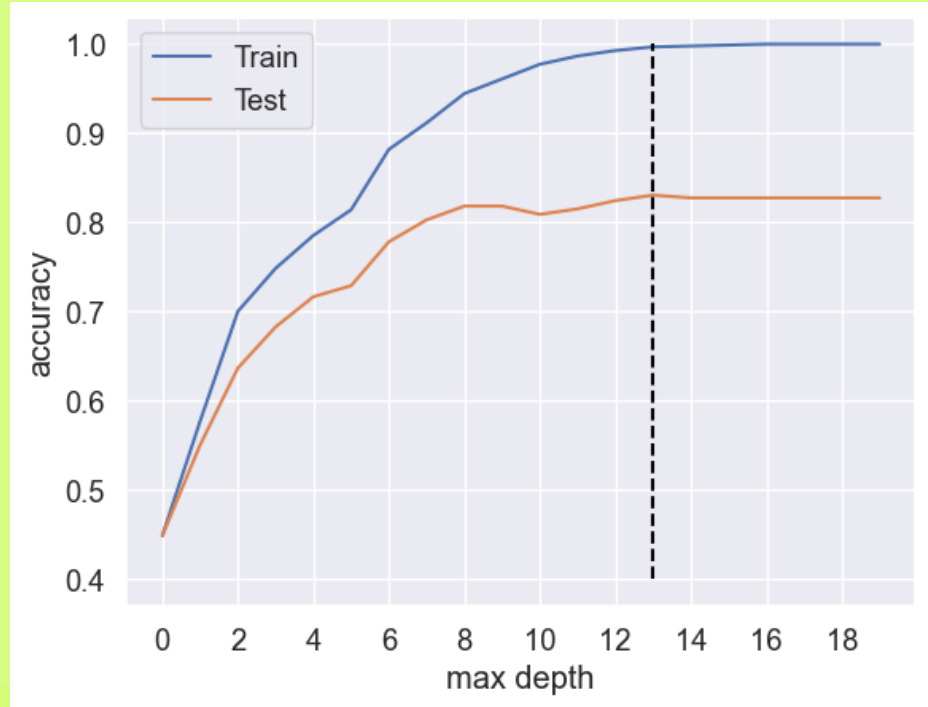
- Training Accuracy = 78.6%
- Testing Accuracy = 71.7%

To improve the model, I tested different max_depth values. The ideal max_depth value was found to be 13.

Updated model parameters:

min_samples=2, max_depth=13

- Training Accuracy = 99.7%
- Testing Accuracy = 83.1%



FURTHER IMPROVEMENT



This is the most performance we can squeeze out this singular decision tree. We could also try different splitting criterion like Gini Impurity to see if it performs better. Instead, the method of improvement I chose to implement was...

Second Model: Random Forest

To improve performance further, I implemented a **Random Forest Classifier** using the previous decision tree as its base.

This Random Forest ensemble method uses **Bootstrap Aggregating (Bagging)** to sample from the training set.

This model has **2 additional parameters**:

- **n_estimators**: The number of decision trees in the forest
- **random_state**: The random state seed for bagging to get reproducible results during testing

```
from collections import Counter

class RandomForest:
    def __init__(self, n_estimators=10, max_depth=4,
                 min_samples=2, random_state=257):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples = min_samples
        self.rng = np.random.default_rng(random_state)

    def fit(self, X, y):
        self.trees = []
        for _ in range(self.n_estimators):
            tree = DecisionTree(max_depth=self.max_depth,
                               min_samples=self.min_samples)
            X_sample, y_sample = self.bootstrap_samples(X, y)
            tree.fit(X_sample, y_sample)
            self.trees.append(tree)

    def bootstrap_samples(self, X, y):
        n_samples = X.shape[0]
        indices = self.rng.choice(n_samples, n_samples,
                                  replace=True)
        return X[indices], y[indices]

    def final_label(self, y):
        counter = Counter(y)
        most_common = counter.most_common(1)[0][0] # the
        most_common_predicted_label
        return most_common

    def predict(self, X):
        preds = np.array([tree.predict(X) for tree in self.
                           trees])
        tree_preds = preds.T
        preds = np.array([self.final_label(pred) for pred in
                           tree_preds])
        return preds
```

Random Forest: Results + Improvement

Base model parameters: min_samples=2, max_depth=13, n_estimators=10, random_state=42

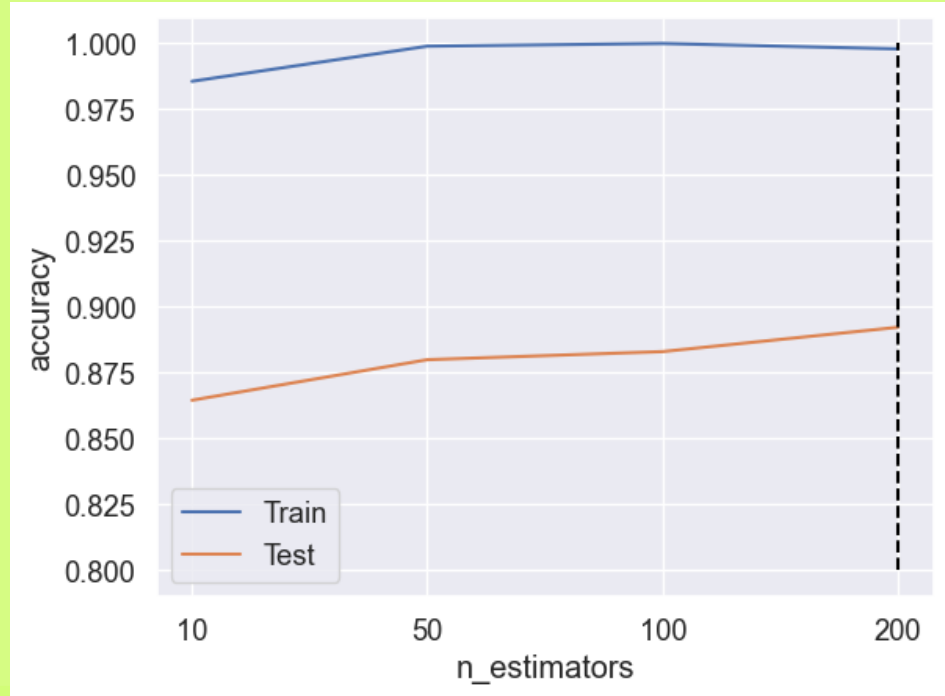
- Training Accuracy = 98.6%
- Testing Accuracy = 86.5%

The ideal n_estimator value was found to be 200.

Updated model parameters:

min_samples=2, max_depth=13, n_estimators=200, random_state=42

- Training Accuracy = 99.8%
- Testing Accuracy = 89.2%



In Comparison

	KNN (k=3)	Support Vector Classifier	Logistic Regression	Random Forest (n_estimators=200)
Training	91.6%	77.8%	65.5%	99.8%
Testing	84.9%	72.9%	68.0%	89.2%
sklearn base models				

Thank you!
